APÉNDICE TÉCNICO: GUÍA DE INICIO RÁPIDO PARA LA CARGA DE DATOS

GameSec Monitor: Telemetría de Servidores y Detección de Incidentes con Python

Este apéndice explica paso a paso los conceptos necesarios para resolver la **Consigna 1 (`carga.py`)**. Se recuerda que la consigna prohíbe terminantemente el uso de librerías externas de análisis (tales como `pandas` o `numpy`); toda la manipulación debe realizarse con la biblioteca nativa `csv` y estructuras básicas de Python (`list` y `dict`).

1. ¿Cómo funciona un archivo CSV en Python estándar?

Un archivo CSV (*Comma-Separated Values*) es un documento de texto plano donde cada fila representa un registro y las columnas están separadas por un delimitador (habitualmente la coma `,`).

Python incluye de fábrica el módulo `csv`:

```
import csv
```

Con la clase `csv.DictReader`, Python lee automáticamente la primera fila (los encabezados) y convierte cada una de las filas siguientes en un diccionario nativo, donde las claves son los nombres de las columnas.

2. El desafío central: Conversión de tipos de datos (Casteo)

Cuando `csv.DictReader` procesa el archivo, todos los valores se leen como cadenas de texto (`str`), sin importar si son números enteros o con decimales.

Por ejemplo, si la fila en el CSV es:

`14,NA-Virginia,User_482,Competitiva,210,120,35.5,5,4`

`csv.DictReader` la interpretará inicialmente como:

```
{
    'id_sesion': '14',
    'servidor': 'NA-Virginia',
    'jugador_id': 'User_482',
    'tipo_partida': 'Competitiva',
    'ping_promedio': '210',
    'fps_promedio': '120',
    'duracion_minutos': '35.5',
    'intentos_fallidos': '5',
    'reportes_trampa': '4'
}
```

Si intentan calcular promedios o comparar condiciones lógicas (por ejemplo, `ping_promedio >200` o sumar la duración), Python arrojará un error de tipo (`TypeError`) o comparará cadenas alfabéticamente en lugar de valores numéricos.

Por ello, al leer cada fila es **obligatorio** castear explícitamente cada campo:

- `int(...)` para contadores y enteros: `id_sesion`, `ping_promedio`, `fps_promedio`, `intentos_fallidos`, `reportes_trampa`.

- `float(...)` para números decimales: `duracion_minutos`.
- **Cadenas de texto**: se mantienen como están (`servidor`, `jugador_id`, `tipo_partida`).

Mapeo de Tipos de Datos

Campo	Tipo Origen (CSV)	Tipo Destino (Python)	Ejemplo de Entrada	Ejemplo de Salida
id_sesion	str	int	'14'	14
servidor	str	str	'NA-Virginia'	'NA-Virginia'
jugador_id	str	str	'User_482'	'User_482'
tipo_partida	str	str	'Competitiva'	'Competitiva'
ping_promedio	str	int	'210'	210
fps_promedio	str	int	'120'	120
duracion_minutos	str	float	'35.5'	35.5
intentos_fallidos	str	int	'5'	5
reportes_trampa	str	int	'4'	4

3. Anatomía de la lectura segura: `with open(...)` y validaciones

Para garantizar que el programa no falle de manera abrupta, deben contemplarse buenas prácticas de manejo de archivos:

1. **Gestor de contexto `with open(...)`**: Garantiza que el archivo se cierre automáticamente en el sistema operativo al terminar de leerse, aun si ocurre un error.
2. **Codificación `encoding='utf-8'`**: Evita problemas de caracteres o tildes en diferentes sistemas operativos (Windows/Linux/macOS).
3. **Validación de existencia**: Con la librería estándar `os`, se comprueba si la ruta realmente existe antes de intentar abrirla (`os.path.exists`).
4. **Validación de contenido**: Si el archivo está vacío o solo contiene encabezados, la lista resultante tendrá longitud 0 y debe notificarse el error mediante `ValueError`.

4. Estructura recomendada para `carga.py`

```python
import csv
import os

def cargar_telemetria(ruta_archivo: str) -> list[dict]:
    """
    Lee el archivo CSV de telemetria, valida su existencia e integridad,
    y transforma cada registro en un diccionario con tipos de datos nativos.

    Parametros:
        ruta_archivo (str): Ruta relativa o absoluta hacia 'gamesec_logs.csv'.

    Retorna:
        list[dict]: Lista de diccionarios, donde cada diccionario representa una sesion.
    """
    # 1. Validar que el archivo exista en la ruta indicada
    if not os.path.exists(ruta_archivo):
        raise FileNotFoundError(f"El archivo '{ruta_archivo}' no existe. Verifique la
            ubicacion.")

    sesiones = []

    # 2. Apertura del archivo con codificacion utf-8
    with open(ruta_archivo, mode='r', encoding='utf-8') as archivo:
        lector = csv.DictReader(archivo)
```

```
24
25          # 3. Recorrer fila por fila y castear los campos numericos
26          for fila in lector:
27              sesion = {
28                  'id_sesion': int(fila['id_sesion']),
29                  'servidor': fila['servidor'],
30                  'jugador_id': fila['jugador_id'],
31                  'tipo_partida': fila['tipo_partida'],
32                  'ping_promedio': int(fila['ping_promedio']),
33                  'fps_promedio': int(fila['fps_promedio']),
34                  'duracion_minutos': float(fila['duracion_minutos']),
35                  'intentos_fallidos': int(fila['intentos_fallidos']),
36                  'reportes_trampa': int(fila['reportes_trampa'])
37              }
38              sesiones.append(sesion)
39
40      # 4. Validar que contenga al menos un registro
41      if len(sesiones) == 0:
42          raise ValueError(f"El archivo '{ruta_archivo}' esta vacio o no contiene registros.")
43
44      return sesiones
```

5. Cómo probar de forma individual que `carga.py` funciona bien

Para no esperar a tener listos `procesamiento.py` y `main.py`, pueden probar directamente su módulo ejecutando `python carga.py` agregando al final del archivo el siguiente bloque de prueba:

```
1  if __name__ == '__main__':
2      archivo_prueba = 'gamesec_logs.csv'
3
4      try:
5          datos = cargar_telemetria(archivo_prueba)
6          print(f"-> Carga exitosa: se leyeron {len(datos)} registros.")
7
8          primer_registro = datos[0]
9          print("\nPrimer registro procesado:")
10         print(primer_registro)
11
12         print("\nVerificacion de tipos de datos:")
13         print("id_sesion es entero?:", isinstance(primer_registro['id_sesion'], int))
14         print("duracion_minutos es float?:", isinstance(primer_registro['duracion_minutos'],
               float))
15         print("ping_promedio es entero?:", isinstance(primer_registro['ping_promedio'], int)
               )
16
17     except Exception as error:
18         print(f"Error al ejecutar la carga: {error}")
```